ALGORITMOS E ESTRUTURAS DE DADOS II

Prof. Alternei Brito

Trabalho Prático I — Árvores Balanceadas

Árvore AVL × Árvore Rubro-Negra

Objetivo geral. Implementar e comparar duas árvores binárias de busca balanceadas — AVL e Rubro-Negra — aplicadas ao mesmo conjunto de registros. O trabalho deve evidenciar o funcionamento das operações de busca, inserção e remoção, os mecanismos de rebalanceamento e os efeitos dessas estratégias sobre a altura e o custo das operações.

1 Descrição do trabalho

Cada equipe deverá desenvolver uma aplicação capaz de armazenar registros de um dos minimundos definidos neste enunciado. A mesma coleção de registros deverá ser mantida em duas estruturas independentes:

- uma **Árvore AVL**;
- uma **Árvore Rubro-Negra**.

Cada registro possuirá uma chave única de ordenação. Essa chave será utilizada em todas as comparações realizadas durante inserções, buscas e remoções. Os demais atributos do registro permanecem associados ao nó, mas não determinam sua posição na árvore.

1.1 Chaves e regra de ordenação

A propriedade de Árvore Binária de Busca deve ser preservada em ambas as estruturas:

chave menor → subárvore esquerda | chave maior → subárvore direita

Chaves textuais ou identificadores numéricos podem ser usados desde que exista uma ordem consistente.

Por exemplo, um ISBN deve ser normalizado, removendo hífens e espaços, e pode ser armazenado como string de 13 dígitos. Como todos possuem o mesmo comprimento, a comparação lexicográfica coincide com a ordem numérica.

2 Requisitos obrigatórios

2.1 Operações comuns às duas árvores

A aplicação deverá implementar:

- inserir um registro;
- buscar um registro pela chave;
- remover um registro pela chave;
- obter o registro de menor chave;
- obter o registro de maior chave;
- realizar caminhamentos em pré-ordem, em ordem e pós-ordem;
- informar a altura atual da árvore;
- informar a quantidade de nós armazenados;
- exibir a estrutura da árvore no terminal de forma compreensível.

2.2 Requisitos específicos da AVL

- manter e atualizar altura ou fator de balanceamento dos nós;
- implementar rotações simples à esquerda e à direita;
- implementar rotações duplas LR e RL;
- rebalancear a árvore após inserções e remoções;
- contabilizar o número de rotações realizadas;
- disponibilizar uma rotina de validação que confirme, para todos os nós, $|FB| \leq 1$ e a propriedade de busca binária.

2.3 Requisitos específicos da Rubro-Negra

- representar a cor de cada nó e tratar adequadamente folhas NIL, conforme a implementação adotada;
- realizar recolorações e rotações necessárias após inserções;
- realizar as correções necessárias após remoções;

- contabilizar separadamente rotações e recolorações;
- disponibilizar uma rotina de validação das propriedades Rubro-Negras: raiz preta; nenhum nó vermelho com filho vermelho; mesma altura negra nos caminhos até NIL; e propriedade de busca binária.

2.4 Modo de execução detalhado

A aplicação deverá possuir uma opção de execução detalhada para mostrar, quando houver rebalanceamento, informações como:

- chave inserida ou removida;
- nó ou região onde ocorreu o desequilíbrio;
- tipo de rotação realizada;
- recolorações efetuadas na Rubro-Negra;
- estrutura da árvore após a correção.

3 Minimundos

Cada equipe deverá trabalhar com um dos minimundos abaixo. O professor poderá distribuir os temas para evitar repetição excessiva. A equipe pode acrescentar atributos, mas não deve substituir a chave definida sem autorização.

1. **Biblioteca**
   Entidade: Livro
   Chave: ISBN-13 normalizado (string com 13 dígitos)
   Atributos mínimos: ISBN, título, autor, ano, editora
   Observação: Ex.: 9780131103627. Remover hífens e espaços antes de comparar.

2. **Sistema Acadêmico**
   Entidade: Aluno
   Chave: Matrícula institucional
   Atributos mínimos: matrícula, nome, curso, período
   Observação: Tratar a matrícula como identificador único; pode ser string ou inteiro, desde que o formato seja uniforme.

3. **Controle de Estoque**
   Entidade: Produto
   Chave: Código do produto
   Atributos mínimos: código, descrição, categoria, quantidade, preço
   Observação: Código numérico único definido no cadastro.

4. **Clínica / Atendimento**
   Entidade: Paciente
   Chave: Número do prontuário
   Atributos mínimos: prontuário, nome, data de nascimento, telefone
   Observação: O prontuário deve ser único e utilizado nas operações da árvore.

5. **Cadastro de Veículos**
   Entidade: Veículo
   Chave: RENAVAM normalizado
   Atributos mínimos: RENAVAM, placa, modelo, ano, proprietário
   Observação: Armazenar o RENAVAM sem pontuação. Recomenda-se string para preservar zeros à esquerda.

6. **Sistema Bancário**
   Entidade: Conta
   Chave: Número da conta
   Atributos mínimos: número da conta, titular, agência, tipo, saldo
   Observação: A chave é o número da conta. O saldo é apenas atributo do registro e não interfere na posição do nó.

4 Interface e fluxo mínimo da aplicação

A interface pode ser textual. Um menu mínimo deverá permitir selecionar a estrutura e executar as operações principais. Exemplo de fluxo:

1. selecionar AVL ou Rubro-Negra;
2. cadastrar registro;
3. buscar por chave;
4. remover por chave;
5. listar registros em ordem;
6. exibir altura e quantidade de nós;

7. exibir estatísticas de rebalanceamento;
8. validar as propriedades da árvore;
9. executar os experimentos automáticos.

5 Experimentos comparativos

As duas estruturas deverão receber exatamente as mesmas sequências de chaves em cada experimento. Os valores de n abaixo são referências; o professor poderá reduzi-los ou ampliá-los conforme o ambiente de execução.

Experimento A — Inserção em ordem crescente

Inserir chaves distintas em ordem crescente, por exemplo com $n = 100$, 1.000 e 10.000. Registrar a altura final, o tempo total de inserção e os eventos de rebalanceamento.

Experimento B — Inserção em ordem aleatória

Gerar as mesmas quantidades de chaves em ordem aleatória e repetir as medições. A semente do gerador aleatório deve ser registrada para que o experimento possa ser reproduzido.

Experimento C — Busca

Após construir as árvores, realizar buscas por chaves existentes e inexistentes. Além do tempo, registrar a quantidade de nós visitados ou comparações realizadas por busca e calcular a média.

Experimento D — Remoção

Remover subconjuntos dos elementos inseridos (sugestão: 10%, 30% e 50%). Registrar a altura antes e depois, tempo de remoção, rotações e, na Rubro-Negra, recolorações.

5.1 Tabela de resultados obrigatória

Tabela 1: Métricas obrigatórias para comparação.

Métrica	AVL	Rubro-Negra
Altura final		
Rotações na inserção		
Rotações na remoção		
Recolorações		
Média de nós visitados na busca		
Tempo total de inserção		
Tempo total de busca		
Tempo total de remoção		

5.2 Análise escrita

O relatório deverá interpretar os resultados e responder, no mínimo:

- Qual estrutura apresentou menor altura nos cenários testados?
- Qual estrutura realizou mais rotações durante inserções e remoções?
- Como as recolorações da Rubro-Negra aparecem nos experimentos?
- A menor altura resultou sempre em menor tempo de execução?
- Como a ordem de inserção influenciou as duas estruturas?
- Em quais operações as diferenças entre AVL e Rubro-Negra ficaram mais evidentes?
- Quais limitações metodológicas existem na comparação realizada?

6 Organização sugerida do código

O projeto deverá ser modularizado. Uma organização recomendada é:

```
projeto/
    avl.py
    rubro_negra.py
    modelos.py
    experimentos.py
    main.py
    README.md
```

As duas estruturas devem possuir interfaces equivalentes sempre que possível, facilitando a execução dos mesmos testes. É recomendável separar o código de experimentação do código das árvores.

7 Entregáveis

- código-fonte completo e executável;
- README com instruções de execução, dependências e identificação do minimundo;
- conjunto de dados ou script que gere os dados usados nos experimentos;
- relatório em PDF com descrição da solução, resultados, tabelas/gráficos e análise comparativa;
- arquivo de resultados brutos (CSV, TXT ou formato equivalente), quando aplicável;
- enviar o link do repositório no Github no Google Classroom.

8 Regras gerais

- As árvores AVL e Rubro-Negra devem ser implementadas pela equipe. Não é permitido utilizar bibliotecas que forneçam diretamente essas estruturas.
- É permitido utilizar bibliotecas auxiliares para medição de tempo, leitura de dados, geração de números aleatórios e produção de gráficos.
- As chaves devem ser únicas. A política para tentativa de inserção duplicada deve ser explicitada no README.
- Os experimentos devem ser reproduzíveis: registre tamanhos, sequências ou sementes aleatórias utilizadas.
- O programa deve lidar de forma segura com busca ou remoção de chave inexistente.
- A equipe deve ser capaz de explicar todo o código entregue durante a defesa.

9 Defesa prática

Na defesa, qualquer integrante poderá ser solicitado a executar e explicar operações na AVL ou na Rubro-Negra. Exemplos de solicitações:

- inserir uma sequência de chaves e indicar a rotação ou recoloração esperada;
- remover uma chave e explicar o rebalanceamento;
- mostrar por que a árvore resultante satisfaz suas propriedades;
- explicar uma função de rotação, busca ou remoção;
- interpretar um dos resultados experimentais;
- realizar pequena alteração no código ou nos testes.

10 Critérios de avaliação

Tabela 2: Critérios de avaliação do trabalho.

Critério	**Valor**
Implementação correta da AVL	2,0
Implementação correta da Rubro-Negra	2,5
Operações, validações e robustez	1,0
Experimentos e coleta das métricas	1,0
Análise comparativa e relatório	1,0
Organização e qualidade do código	0,5
Defesa prática	2,0
Total	**10,0**

11 Checklist antes da entrega

☐ AVL permanece válida após inserções e remoções.

☐ Rubro-Negra permanece válida após inserções e remoções.

☐ Busca, mínimo, máximo e caminhamentos foram testados.

☐ Rotações e recolorações são contabilizadas.

☐ Os dois métodos recebem os mesmos dados nos experimentos.

☐ Resultados são reproduzíveis.

☐ Relatório apresenta tabelas/gráficos e interpretação.

☐ README contém instruções suficientes para executar o projeto.

O objetivo do trabalho não é apenas produzir duas implementações funcionais, mas compreender e demonstrar como diferentes estratégias de balanceamento afetam a estrutura e o comportamento das árvores.

Data de entrega: 17 de outubro de 2026.